

SPEEDCUBING IITK
2026-27[SUMMER]

Image Manipulation and Resizing

-CUBE CANVAS
JEETENDAR MEENA[240495]

IMAGE CROPPING

cropping means extracting a specific part of an image.

```
cropped = image.crop((x1, y1, x2, y2))
```

Mathematically:

$$I_c(x, y) = I(x + x_0, y + y_0)$$

where:

(x_0, y_0) = starting point (crop origin)

REGION OF INTEREST (ROI)

The selected cropped area is called ROI.

Benefits:

- Reduces computation
- Removes unwanted parts
- Focuses processing on important region

ASPECT RATIO

Aspect Ratio = Width / Height

Example:

1920 × 1080

Aspect Ratio = 1920 / 1080 = 1.777 ≈ 16 : 9

IMPORTANCE :

- Maintains proper shape of image
- Prevents stretching or distortion

IMAGE RESIZING

UPSAMPLING (INCREASING SIZE)

- Increasing image size

Example:

100 × 100 → 1000 × 1000

- New pixels are created
- Values must be estimated (interpolation)

DOWNSAMPLING (REDUCING SIZE)

- **Decreasing image size**

Example:

1000 × 1000 → 100 × 100

- Pixels are removed
- Some information is lost

- **CubeCanvas mainly uses downsampling before creating the mosaic.**

INTERPOLATION TECHNIQUES

Interpolation = estimating pixel values when resizing an image

1. NEAREST NEIGHBOR

Nearest Neighbor Interpolation assigns the value of the nearest pixel to the new (resized) pixel location.

Formula:

$$I'(x, y) = I(\text{round}(x), \text{round}(y))$$

Steps:

1. Take the required floating position (x, y)
2. Round x and y to nearest integers
3. Pick the pixel value at that location
4. Assign it directly to output image

NUMERICAL EXAMPLE

GIVEN IMAGE MATRIX:

10	20	30	
40	50	60	
70	80	90	

Find value at:
(x, y) = (1.6, 2.3) ?

STEP 1: ROUND COORDINATES

$$x = 1.6 \rightarrow 2$$

$$y = 2.3 \rightarrow 2$$

So:

$$I'(1.6, 2.3) = I(2, 2)$$

STEP 2: PICK VALUE FROM MATRIX

$$I(2, 2) = 90$$

FINAL ANSWER:

$$I'(1.6, 2.3) = 90$$

BILINEAR INTERPOLATION

method of estimating pixel values
by using a weighted average of the four nearest neighboring pixels.

It produces smoother images compared to nearest neighbor
interpolation
by considering both horizontal and vertical directions.

FORMULA

$$f(x, y) = \sum w_i p_i$$

EXAMPLE:

Given 2×2 pixel block:

100 150
200 250

Let:

$$x = 0.3, y = 0.6$$

So:

$$dx = 0.5 \rightarrow (1-dx) = 0.5$$

$$dy = 0.5 \rightarrow (1-dy) = 0.5$$

STEP 1: CALCULATE WEIGHTS

So all weights:

$$I(0,0): (1-dx)(1-dy) = 0.5 \times 0.5 = 0.25$$

$$I(1,0): (dx)(1-dy) = 0.5 \times 0.5 = 0.25$$

$$I(0,1): (1-dx)(dy) = 0.5 \times 0.5 = 0.25$$

$$I(1,1): (dx)(dy) = 0.5 \times 0.5 = 0.25$$

STEP 2: APPLY FORMULA

$$I' = 0.25 \times 100 + 0.25 \times 150 + 0.25 \times 200 + 0.25 \times 250$$

STEP 3: CALCULATE

$$= 25 + 37.5 + 50 + 62.5$$

$$= 175$$

FINAL ANSWER

$$I' = 175$$

KEY IDEA

When $dx = dy = 0.5$:

→ Point is EXACTLY at center

→ All pixels contribute equally

BICUBIC INTERPOLATION

is a method of estimating pixel values using 16 nearest neighboring pixels (4×4 grid).

FORMULA IDEA:

$I'(x, y)$ is computed using a weighted combination of 16 pixels based on cubic functions (row-wise and column-wise weighting).

EXAMPLE (4×4 NEIGHBORHOOD):

Given pixel block:

	10	20	30	40	
	50	60	70	80	
	90	100	110	120	
	130	140	150	160	

STEP IDEA:

1. Take 16 surrounding pixels
2. Apply cubic weighting in x-direction
3. Apply cubic weighting in y-direction
4. Combine results

LANCZOS INTERPOLATION

$$\mathbf{I}'(\mathbf{x}) = \sum \mathbf{I}(\mathbf{i}) \times \mathbf{L}(\mathbf{x} - \mathbf{i})$$

Used when kernel is already normalized

FORMULA IDEA:

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

LANCZOS KERNEL:

$$L(x) = \text{sinc}(x) \cdot \text{sinc}\left(\frac{x}{a}\right)$$

where:

a = window size (commonly 2 or 3)

Lanczos interpolation is a high-quality image resizing method used in CubeCanvas.

It is based on the sinc function and provides very sharp and smooth results with minimal aliasing.

EXAMPLE IDEA:

Suppose we want to estimate a pixel value using surrounding pixels:

Given pixels:

Position: -1 0 1 2

Values: 100 150 200 250

Find value at:

x = 0.5

Step 1:

Compute distance of each pixel from target point

LANCZOS KERNEL

Step 2:

Apply Lanczos kernel weights:

- **closer pixels → higher weight**
- **farther pixels → lower weight (sinc decay)**

$$L(x) = \text{sinc}(x) \times \text{sinc}(x/a)$$

Let $a = 2$ (Lanczos-2)

Step 3:

Compute weighted sum:

For each pixel i :

$$w(i) = L(x - i)$$

So:

$$w(-1) = L(1.5)$$

$$w(0) = L(0.5)$$

$$w(1) = L(-0.5)$$

$$w(2) = L(-1.5)$$

FINAL RESULT:

$$I' = [\sum I(i) \times w(i)] / [\sum w(i)] \quad U$$

Lanczos weights (sinc-based) do NOT always sum to 1

- So normalization is REQUIRED

SPEEDCUBING IITK
2026-27[SUMMER]

Color Quantization and Distorting

INTRODUCTION

A standard RGB image can contain millions of colors.

Each RGB channel (Red, Green, Blue) can take 256 possible values.

Total number of colors:

$256 \times 256 \times 256 = 256^3 = 16,777,216 \approx 16.7$ million colors

A Rubik's Cube supports only a limited number of colors (typically 6 colors).

COLOR QUANTIZATION

Color Quantization is the process of reducing the number of distinct colors in an image by mapping original colors to a limited predefined palette

16.7 Million Colors → 6 Rubik Cube Colors

RUBIK CUBE COLOR PALETTE

palette = [(255,255,255), (255,255,0), (255,0,0), (255,136,0), (0,0,255), (0,170,0)]	COLOR REPRESENTATION
	(255,255,255) → White
	(255,255,0) → Yellow
	(255,0,0) → Red
	(255,136,0) → Orange[(255,165,0) Your value is slightly darker/more reddish]
	(0,0,255) → Blue
	(0,170,0) → Green[Pure green would be: (0,255,0) version is darker (often used to match real cube col)]

RGB COLOR SPACE GEOMETRY

RGB colors can be represented as points in a 3-dimensional coordinate system.

AXES DEFINITION

X-axis → Red (R)
Y-axis → Green (G)
Z-axis → Blue (B)

Each color is a point in this 3D space:
(R, G, B)

EXAMPLE:

Red → (255, 0, 0)

Closer points → more similar colors

EUCLIDEAN DISTANCE

Let:

$c = (R, G, B) \rightarrow$ pixel color

$p = (r, g, b) \rightarrow$ palette color

Distance between c and p is:

$$d = \sqrt{[(R - r)^2 + (G - g)^2 + (B - b)^2]}$$

EXAMPLE

Given:

$c = (250, 20, 20)$

Compare with:

Red = (255, 0, 0)

White = (255, 255, 255)

DISTANCE FROM RED

$$\begin{aligned} d^2_r &= (250 - 255)^2 \\ &+ (20 - 0)^2 + (20 - 0)^2 \\ &= (-5)^2 + 20^2 + 20^2 \\ &= 25 + 400 + 400 \\ &= 825 \end{aligned}$$

DECISION RULE

- Compute distance from pixel to each palette color
- Select the palette color with minimum distance

That is:

Choose p such that d is minimum

DISTANCE FROM WHITE

$$\begin{aligned} d^2_w &= (250 - 255)^2 + (20 - 255)^2 + (20 - 255)^2 \\ &= (-5)^2 + (-235)^2 + (-235)^2 \\ &= 25 + 55225 + 55225 \\ &= 110475 \end{aligned}$$

COMPARISON

$$825 < 110475$$

FINAL RESULT

Nearest color = Red

Therefore:

Pixel (250, 20, 20) $\rightarrow$ Red

PROBLEM OF SIMPLE QUANTIZATION

when we directly reduce colors (e.g., from millions → 6 colors), the image loses smooth transitions.

This creates unwanted visual defects called ARTIFACTS. It is not part of the original image, but appears due to processing.

Examples of artifacts:

- Blocky edges
- Noise
- Color banding (most important here)

WHAT IS A GRADIENT?

A gradient is a smooth transition between colors or intensities.

Example (grayscale gradient):

Black → Dark Gray → Gray → Light Gray → White

Numerically:

0 → 50 → 100 → 150 → 200 → 255

This change is continuous and smooth

Original gradient:
Smooth curve

HOW DITHERING HELPS

Suppose we reduce the gradient to only 3 levels:

Original:

0 → 50 → 100 → 150 → 200 → 255

After quantization:

0 → 0 → 128 → 128 → 255 → 255

Now:

- Many nearby values become the same
- Smooth change is lost

COLOR BANDING

This creates visible "bands" instead of smooth transitions.

Instead of gradual change, we see:

Flat regions → sudden jump → flat regions

This effect is called COLOR BANDING.

DITHERING

Dithering is used to reduce banding artifacts caused by simple color quantization.

Instead of treating each pixel independently, the quantization error is distributed to neighboring pixels.

This helps in preserving the overall visual appearance of the image

FLOYD-STEINBERG DITHERING

1. Take current pixel
2. Quantize it (map to nearest palette color)
3. Compute error
4. Distribute error to neighboring pixels
5. Move to next pixel (scan left → right, top → bottom)

QUANTIZATION ERROR

Let:

Old = original pixel value

New = quantized pixel value

Error = Old - New

EXAMPLE

Old pixel:

(120, 120, 120)

After quantization:

New = (100, 100, 100)

Error:

Error = (120-100, 120-100, 120-100)

= (20, 20, 20)

ERROR DIFFUSION MATRIX (FLOYD-STEINBERG)

The Floyd-Steinberg algorithm distributes quantization error to neighboring pixels using fixed weights.

Matrix representation:

* 7/16
3/16 5/16 1/16

where:

* = current pixel being processed

IMPORTANT PROPERTY

$$7/16 + 3/16 + 5/16 + 1/16 = 16/16 = 1$$

This means:

- The total error is fully preserved
- No information is lost, only redistributed

QUANTIZATION ERROR

Error = (20, 20, 20)

Right pixel:

= $(20 \times 7/16, 20 \times 7/16, 20 \times 7/16)$
= (8.75, 8.75, 8.75)

Bottom-left pixel:

= $(20 \times 3/16, 20 \times 3/16, 20 \times 3/16)$
= (3.75, 3.75, 3.75)

Bottom pixel:

= $(20 \times 5/16, 20 \times 5/16, 20 \times 5/16)$
= (6.25, 6.25, 6.25)

Bottom-right pixel:

= $(20 \times 1/16, 20 \times 1/16, 20 \times 1/16)$
= (1.25, 1.25, 1.25)

FINAL UPDATE RULE

Each neighboring pixel gets:

New value = Old value + distributed error

ERROR PROPAGATION EQUATIONS

Let:

e = error at pixel (x, y)

Then updates are:

$$I(x+1, y) = I(x+1, y) + (7/16) * e$$

$$I(x-1, y+1) = I(x-1, y+1) + (3/16) * e$$

$$I(x, y+1) = I(x, y+1) + (5/16) * e$$

$$I(x+1, y+1) = I(x+1, y+1) + (1/16) * e$$

INTERPRETATION

- Right pixel gets largest share (7/16)
- Bottom pixel gets next (5/16)
- Bottom-left gets smaller (3/16)
- Bottom-right gets smallest (1/16)

WHY DITHERING WORKS

The human eye performs spatial averaging.

This means:

- It blends nearby pixels visually
- Small variations are averaged out

As a result:

- Distributed errors create illusion of missing colors
- Gradients appear smooth even with limited palette

PIXEL CLIPPING

pixel values are modified
by adding quantization error.

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & 0 \leq x \leq 255 \\ 255, & x > 255 \end{cases}$$

This may produce invalid RGB values.

Pixel clipping ensures that all values stay within valid limits.

Each pixel value is restricted to the range [0, 255].

Let:

N = number of pixels

K = number of colors in palette

COMPLEXITY ANALYSIS

Complexity means how much time or memory an algorithm
needs
to run as the input size increases.

For each pixel:

- Compare with all K colors

- Time = O(K)

For whole image:

- Total time = O(N × K)

Since K = 6 (constant):

O(N × 6) = O(N)

FINAL:

Algorithm is linear and very efficient.

SPEEDCUBING IITK
2025-26[SUMMER]

Image Enhancement Techniques

Brightness Adjustment

What is Brightness?

Brightness controls how light or dark an image appears.
It changes ALL pixel values uniformly.

Mathematical Model

,

$$I'(x, y) = kI(x, y)$$

Where:

- $I(x, y)$ = original pixel value
- $I'(x, y)$ = new pixel value
- k = brightness factor

Interpretation

$k > 1 \rightarrow$ brighter image

$k < 1 \rightarrow$ darker image

$k = 1 \rightarrow$ no change

Example (Grayscale)

Given:

$$I = 100$$

Case 1: $k = 2$

$$I' = 2 \times 100 = 200 \rightarrow \text{brighter}$$

Case 2: $k = 0.5$

$$I' = 0.5 \times 100 = 50 \rightarrow \text{darker}$$

RGB Case

Brightness affects all channels equally.

Given:

$$\text{RGB} = (100, 150, 200)$$

$$k = 1.5$$

Apply to each channel:

$$R' = 1.5 \times 100 = 150$$

$$G' = 1.5 \times 150 = 225$$

$$B' = 1.5 \times 200 = 300$$

Result before clipping:

$$(150, 225, 300)$$

Clipping

Pixel values must stay in range $[0, 255]$

So:

$$(150, 225, 300) \rightarrow (150, 225, 255)$$

Brightness = scaling intensity values

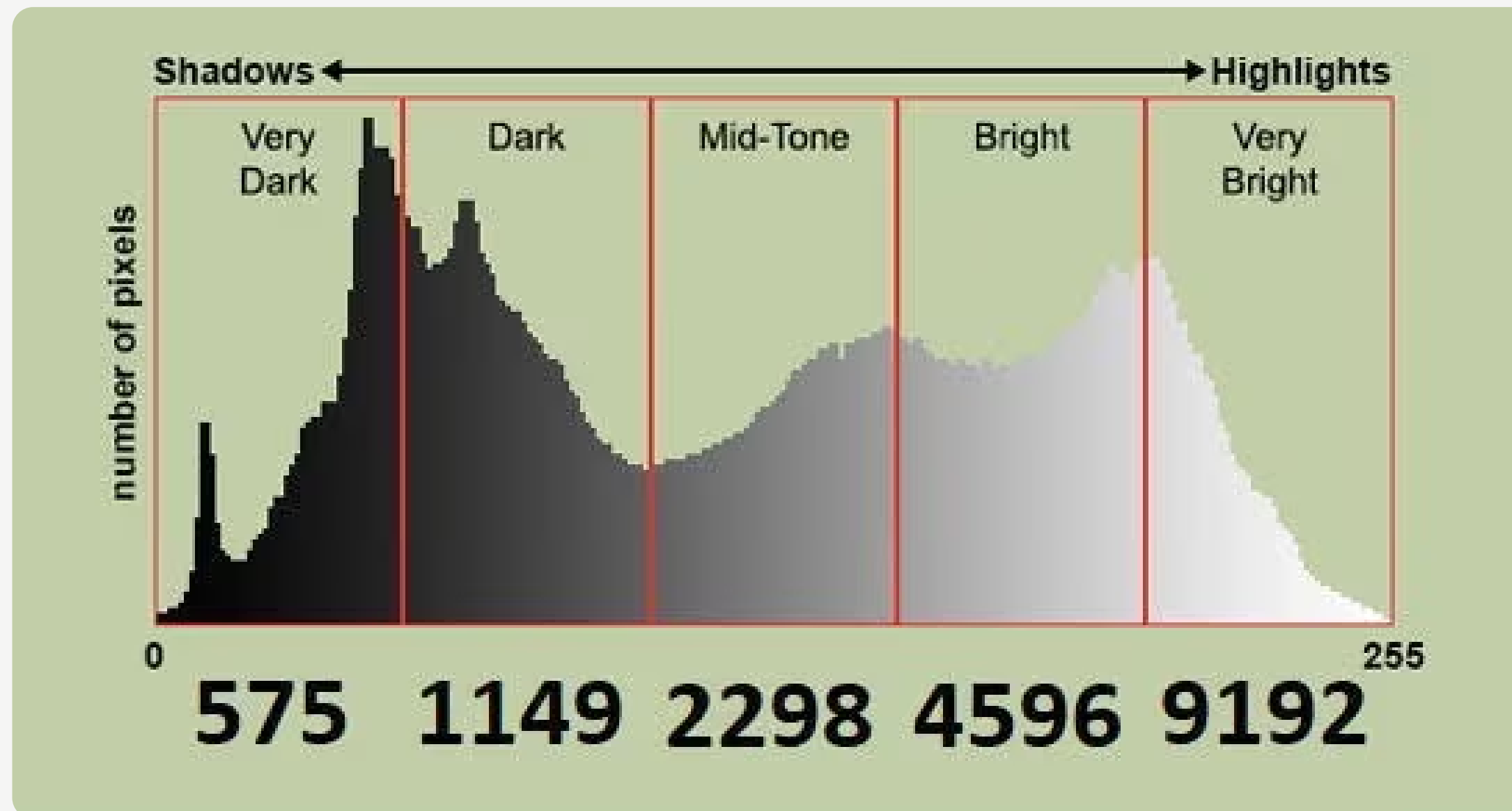
- multiply all pixels by k
- same effect on entire image
- no change in contrast or color ratio

Effect on Histogram :

Brightness does NOT change shape of histogram
It **only shifts it**

→ only overall lightness changes

Example



Contrast Enhancement

Contrast is defined as the degree to which pixel intensities differ from a reference level (usually the mean or midpoint intensity).

Mathematically:

$$\text{Contrast} \propto (\text{Intensity} - \text{Mean Intensity})$$

To modify contrast, we scale this deviation:

$$\text{New deviation} = \alpha \times (\text{old deviation})$$

Derivation of Contrast Adjustment Formula

1. Contrast definition:

Contrast measures how far pixel intensities are from the midpoint (average intensity).

For an 8-bit image:

Intensity range = 0 to 255

Midpoint ≈ 128

Deviation from midpoint:

$$d = I - 128$$

2. Scale the deviation:

To adjust contrast, multiply deviation by a factor α :

$$\text{new deviation} = \alpha(I - 128)$$

3. Shift back to valid intensity range:

Add midpoint again to return to image scale:

$$I' = \alpha(I - 128) + 128$$

4. Interpretation:

$\alpha > 1 \rightarrow$ increases contrast (values spread away from 128)

$\alpha < 1 \rightarrow$ decreases contrast (values move toward 128)

$\alpha = 1 \rightarrow$ no change

5. General form:

New intensity = mean + α (old intensity – mean)

For 8-bit image:

$$I' = 128 + \alpha(I - 128)$$

Example 1: Increase contrast ($\alpha = 1.5$)

Take pixel values:

$I = 100$ (darker pixel)

$I = 200$ (brighter pixel)

For $I = 100$:

$$\begin{aligned} I' &= 128 + 1.5(100 - 128) \\ &= 128 + 1.5(-28) \\ &= 128 - 42 \\ &= 86 \end{aligned}$$

For $I = 200$:

$$\begin{aligned} I' &= 128 + 1.5(200 - 128) \\ &= 128 + 1.5(72) \\ &= 128 + 108 \\ &= 236 \end{aligned}$$

Result:

$100 \rightarrow 86$ (darker)

$200 \rightarrow 236$ (brighter)

Contrast increased (values move away from 128)

What is a Histogram (in images)?

A histogram is just a count of how many pixels have each brightness value.

Think of it like this:

- X-axis → brightness (0 = black, 255 = white)
- Y-axis → number of pixels

Example:

If many pixels have value 120 → histogram is tall at 120

If few pixels have value 200 → histogram is small at 20

Before Contrast Stretching:

Pixel values: 100 → 150 (very close range)

Histogram:

All bars are packed between 100 and 150

Meaning:

- Image looks dull
- No strong dark or bright areas
- Everything looks similar

After Contrast Stretching:

Pixel values: 50 → 200 (wide range)

Histogram:

Bars spread from 50 to 200

Meaning:

- Dark pixels become darker
- Bright pixels become brighter
- Image looks clearer and sharper

Key Idea :

Histogram shows how spread out pixel values are.

Narrow histogram → low contrast (flat image)

Wide histogram → high contrast (clear image)

What is Saturation?

Saturation measures how far a color is from gray.

- High saturation → vivid, strong colors
- Low saturation → dull, grayish colors

Key idea:

Gray means $R = G = B$

Grayscale Conversion

$$\text{Gray} = 0.299R + 0.587G + 0.114B$$

- This gives the brightness (neutral version of the color)
- Human eye is most sensitive to green, so G has highest weight

Saturation Formula

Where:

- I = original color (R, G, B)
- Gray = grayscale value
- s = saturation factor

$$I' = \text{Gray} + s(I - \text{Gray})$$

Meaning Of Formula:

$I - \text{Gray} \rightarrow$ distance from gray (color intensity)

So:

New color = Gray + (scaled distance from gray)

Cases :

$s = 0$:

$I' = \text{Gray}$

→ **completely grayscale image**

$s = 1$:

$I' = I$

→ **original image (no change)**

$s > 1$:

→ colors move away from gray

→ **more vibrant**

$s < 1$:

→ colors move toward gray

→ **more dull**

Example

Given:

RGB = (200, 50, 50)

Step 0: Compute Gray

Gray = $0.299(200) + 0.587(50) + 0.114(50)$
 ≈ 95

Case 1: s = 0

I' = (95, 95, 95)

→ grayscale image

Case 2: s = 2

Step 1: Difference from gray

(200-95, 50-95, 50-95)

= (105, -45, -45)

Step 2: Multiply by s

(210, -90, -90)

Step 3: Add Gray back

(95+210, 95-90, 95-90)

= (305, 5, 5)

Final (clipped to 0-255):

(255, 5, 5)

→ very strong red (high saturation)

What is Sharpness?

Sharpness controls how clear edges appear in an image.

Edge = sudden change in intensity

Example:

10, 10, 10, 250, 250, 250

→ big jump from 10 to 250 = edge

More sharpness → edges become more visible

Why edges matter

Used in:

- Object detection
- Shape recognition
- Feature extraction
- Human visual perception

Unsharp Masking Formula

Where:

- Original = input image
- Blurred = smoothed version (edges removed)
- k = sharpening strengt



Step-by-step process

Step 1: Blur the image

→ removes edges (keeps smooth areas)

Step 2: Find edges

Edge = Original – Blurred

Step 3: Add edges back

Enhanced = Original + k × Edge

Why this works

Blur removes high-frequency (edges)

So:

Original – Blurred = only edges

Adding this back:

→ increases edge contrast

→ image looks sharper

numerical example

Original: 10, 10, 10, 250, 250, 250

Blurred: 10, 10, 80, 180, 250, 250 (smoothed)

Edge = Original – Blurred:

0, 0, -70, 70, 0, 0

Now add edges ($k = 1$):

Sharpened:

$10+0, 10+0, 10-70, 250+70, 250+0, 250+0$

$= 10, 10, -60, 320, 250, 250$

After clipping (0–255):

$= 10, 10, 0, 255, 250, 250$

→ edge becomes much stronger

What is Convolution?

Convolution is a process where we slide a small matrix (called kernel/filter) over an image and compute values by multiplying and adding.

In simple words:

"Take a small window → multiply corresponding values → add them"

General Equation

$$G(x, y) = \sum \sum f(i, j) \cdot h(x - i, y - j)$$

Where:

- $f(i, j)$ = original image pixel
- h = kernel (filter matrix)
- $G(x, y)$ = output pixel

(Simple idea: weighted sum of neighbors)

What is a Kernel?

A kernel is a small matrix (filter) used to process an image.

It slides over the image and performs:

→ multiply + add operation

So:

Kernel = "operation tool"

How Convolution Works (Step-by-step)

Step 1:

Place kernel over a pixel (center position)

Step 2:

Multiply each kernel value with corresponding pixel

Step 3:

Add all values

Step 4:

Replace center pixel with result

Step 5:

Slide kernel across entire image

Numerical Example

Given Image (3×3):

	1	2	3	
	4	5	6	
	7	8	9	

Given Kernel (2×2):

	1	0	
	0	-1	

PROCESS (SLIDING WINDOW)

Kernel moves like this:

Step positions:

- (1) Top-left
- (2) Top-right
- (3) Bottom-left
- (4) Bottom-right

CALCULATIONS

Step 1: Top-left region

Region:

	1	2	
	4	5	

Multiply + Add:

$$\begin{aligned} &= (1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times (-1)) \\ &= 1 + 0 + 0 - 5 \\ &= -4 \end{aligned}$$

Step 2: Top-right region

Region:

	2	3	
	5	6	

Calculation:

$$\begin{aligned} &= (2 \times 1) + (3 \times 0) + (5 \times 0) + (6 \times (-1)) \\ &= 2 + 0 + 0 - 6 \\ &= -4 \end{aligned}$$

Step 3: Bottom-left region

Region:

	4	5	
	7	8	

Calculation:

$$\begin{aligned} &= (4 \times 1) + (5 \times 0) + (7 \times 0) + (8 \times (-1)) \\ &= 4 + 0 + 0 - 8 \\ &= -4 \end{aligned}$$

Step 4: Bottom-right region

Region:

	5	6	
	8	9	

Calculation:

$$\begin{aligned} &= (5 \times 1) + (6 \times 0) + (8 \times 0) + (9 \times (-1)) \\ &= 5 + 0 + 0 - 9 \\ &= -4 \end{aligned}$$

5. FINAL OUTPUT :

	-4	-4	
	-4	-4	

- Multiply corresponding elements
- Add all results
- Slide kernel over image
- Output size reduces (if no padding)

What is Gaussian Blur?

Gaussian Blur is a smoothing technique.

It:

- reduces noise
- removes small details
- makes image softer

Instead of treating all neighbors equally,
Gaussian blur gives:

- more importance to nearby pixels
- less importance to far pixels

→ produces natural smooth blur

Gaussian Kernel

Gaussian Kernel (3×3 example)

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- center (4) has highest weight
- corners (1) have lowest weight

Gaussian Function/Distribution

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

where:

$x, y \rightarrow$ distance from center

σ (sigma) $\rightarrow$ controls blur amount

Bigger $\sigma \rightarrow$ more blur

Smaller $\sigma \rightarrow$ less blur

Steps

Step 1:
Place kernel over image

Step 2:
Multiply each pixel with kernel value

Step 3:
Add all values

Step 4:
Divide by 16 (normalization)

Step 5:
Move to next pixel

Example

Image:

[10 10 10
10 50 10
10 10 10]

Apply kernel:

$$\begin{aligned} &= (1 \times 10 + 2 \times 10 + 1 \times 10 \\ &\quad + 2 \times 10 + 4 \times 50 + 2 \times 10 \\ &\quad + 1 \times 10 + 2 \times 10 + 1 \times 10) / 16 \end{aligned}$$

$$\begin{aligned} &= (10 + 20 + 10 + 20 + 200 + 20 + 10 + 20 + 10) / 16 \\ &= 320 / 16 \\ &= 20 \end{aligned}$$

New center pixel = 20

→ sharp peak (50) becomes smoother (20)

Effects of Gaussian Blur

Advantages:

- Removes noise
- Smooths textures
- Reduces small details
- Creates soft look

Disadvantages:

- Reduces sharpness
- Blurs edges

- **Average filter → equal weight**
 - **Gaussian → weighted average**
- So Gaussian gives better, natural blur
→ smooth + natural looking blur**

Frequency Domain Interpretation

What does “frequency” mean in images?

Frequency = how fast pixel values change

- Slow change → low frequency
- Fast change → high frequency

Low Frequencies

These represent smooth parts of the image:

- Gradual color/intensity changes
- Large uniform regions (sky, wall)
- Overall structure of objects

Example:

10, 12, 14, 16, 18 → slow change → low frequency

High Frequencies

These represent sudden changes:

- Edges (object boundaries)
- Fine details (texture, hair, patterns)
- Noise (random variation)

Example:

10, 10, 10, 250, 250 → sudden jump → high frequency

Gaussian Blur in Frequency Terms

Gaussian Blur = Low Pass Filter

Meaning:

- Keeps low frequencies (smooth areas)
- Removes high frequencies (edges, noise)

Gaussian Blur removes high-frequency content

- keeps smooth regions
- removes edges and noise
- acts as a Low Pass Filter

Image Flipping

What is Image Flipping?

Flipping means reversing the image along an axis.

Two types:

- | | |
|----------------------------------|---------------------------------------|
| • Horizontal Flip → left ↔ right | mirror image (like looking in mirror) |
| • Vertical Flip → top ↔ bottom | upside down image |

Horizontal Flip

Operation:

`ImageOps.mirror(img)`

Transformation:

$x' = W - 1 - x$ Where:

- x = original x-position
- W = image width
- x' = new x-position

Meaning:

- pixel from left moves to right
- pixel from right moves to left

Example:

Original:

A B C

After Horizontal Flip:

C B A

Vertical Flip

Operation:

`ImageOps.flip(img)`

Transformation:

$y' = H - 1 - y$ Where:

- y = original y-position
- H = image height
- y' = new y-position

Meaning:

- top becomes bottom
- bottom becomes top

Example:

Original:

A
B
C

After Vertical Flip:

C
B
A

Flipping does NOT change pixel values
It only changes their positions

- Horizontal: reverse columns
- Vertical: reverse rows

Order of Operations

Operations are applied step-by-step in this order:

Brightness



Contrast



Saturation



Sharpness



Blur



Flip

Why order matters

Image operations are NOT interchangeable.

Changing order → different final result

Example:

If you blur first and then sharpen:
→ sharpening tries to recover lost edges

If you sharpen first and then blur:
→ blur will reduce sharpness again

Computational Complexity

1. Brightness $\rightarrow O(N)$

- Each pixel is processed once
- Just add or subtract a value

Total work $\propto N$

2. Contrast $\rightarrow O(N)$

- Apply formula to each pixel:
 $I' = 128 + \alpha(I - 128)$
- One calculation per pixel

Total work $\propto N$

3. Saturation $\rightarrow O(N)$

- Compute grayscale + adjust each pixel
- Constant work per pixel

Total work $\propto N$

4. Sharpness $\rightarrow O(N)$

- Each pixel updated using neighbors
- Small fixed kernel (like 3×3)

Work per pixel is constant

Total work $\propto N$

5. Gaussian Blur $\rightarrow O(N \times k^2)$

Where:

k = kernel size (e.g., 3, 5, 7)

Reason:

- For each pixel $\rightarrow$ apply $k \times k$ kernel
- That means k^2 operations per pixel

Total work $\propto N \times k^2$

SPEEDCUBING IITK
2025-26[SUMMER]

PDF Export and Mosaic Generation

INTRODUCTION

After image processing is completed, the final mosaic is converted into printable instructions.

Large mosaics are difficult to assemble manually.

So the image is divided into smaller sections for ease of construction.

BENEFITS:

- Easier assembly
- Reduced confusion
- Better organization

MOSAIC PARTITIONING

The full mosaic image is divided into smaller chunks.

CHUNK SIZE:

9 squares

REASON:

Each chunk corresponds to a 3×3 Rubik Cube block

COORDINATE LABELING

To make mosaic sections easy to identify, each section is labeled using a coordinate system.

COLUMN LABELS:

A, B, C, D, E, F, ...

ROW LABELS:

1, 2, 3, 4, 5, 6, ...

Coordinates convert the mosaic into a structured grid system for easy identification and assembly

COMPLETE WORKFLOW (CUBECANVAS)

